D

管程（已过时）

在并发编程开始变得重要的那段时间，面向对象编程也在流行起来。毫不奇怪，人们开始考虑怎样把同步融入一种更结构化的编程环境。

由此出现的一种方法就是**管程**（monitor）。管程最早由 Per Brinch Hansen 描述 [BH73]，后来又经 Tony Hoare 完善 [H74]，它背后的思想相当简单。来看下面这个用 C++ 写法写成的假想管程：

注意：这是一个“假想的”类，因为 C++ 并不支持管程，所以 `monitor` 关键字并不存在。不过，Java 确实支持管程，用的是所谓的**同步**（synchronized）方法。下面我们既会看看怎样在 C/C++ 中做出和管程很像的东西，也会看看怎样使用 Java 的同步方法。

在这个例子中，你可能注意到了我们的老朋友：账户，以及往余额里存钱和取钱的例程。你可能还注意到，这些都是**临界区**（critical section）；如果多个线程同时调用它们，就会出现竞态条件（race condition，也叫竞争条件），可能得到错误的结果。

不过，在管程类里你不会遇到麻烦，因为管程保证**任一时刻只有一个线程能在管程中活动**。因此，上面的例子是一段完全安全、能正常工作的代码；多个线程可以调用 deposit() 或 withdraw()，并且清楚互斥（mutual exclusion）是有保证的。

管程是怎么做到这一点的？很简单：用一把锁。每当某个线程试图调用管程的例程时，它都会隐式地尝试获取管程锁。如果成功，它就能调用这个例程，运行方法里的代码；如果失败，它就会阻塞，直到管程里的那个线程把手头的事做完。因此，如果我们写一个下面这样的 C++ 类，就能实现和上面的管程类完全相同的目标：

```
monitor class account {
  private:
    int balance = 0;

  public:
    void deposit(int amount) {
        balance = balance + amount;
    }
    void withdraw(int amount) {
        balance = balance - amount;
    }
};
```

图 D.1：**一个假想的管程类**

```
class account {
  private:
    int balance = 0;
    pthread_mutex_t monitor;

  public:
    void deposit(int amount) {
        pthread_mutex_lock(&monitor);
        balance = balance + amount;
        pthread_mutex_unlock(&monitor);
    }
    void withdraw(int amount) {
        pthread_mutex_lock(&monitor);
        balance = balance - amount;
        pthread_mutex_unlock(&monitor);
    }
};
```

图 D.2：**一个表现得像管程的 C++ 类**

从这个例子可以看出，管程并没有自动替你做太多事。说到底，它只是获取一把锁、再释放它。这样做，我们就满足了管程的要求：正如所期望的，任一时刻只有一个线程能在 deposit() 或 withdraw() 中活动。

D.1 何必用管程?

你可能会好奇，为什么要发明管程，而不直接显式加锁。当时，面向对象编程刚刚兴起。所以，当时的想法是把并发编程中的一些关键概念，优雅地融入面向对象的一些基本做法中。仅此而已。

D.2 除了自动加锁，还有别的好处吗?

言归正传。我们在讨论信号量时已经知道，光有锁还不够；例如，为了实现生产者/消费者方案，我们之前用信号量来做两件事：一是在等待某个条件发生变化时让线程睡眠（例如，生产者等待缓冲区被清空），二是在某个条件发生变化后唤醒线程（例如，消费者发信号表示它确实已经清空了一个缓冲区）。

管程通过一种叫**条件变量**（condition variable）的显式结构来支持这种功能。我们来看一下生产者/消费者方案，这次用管程和条件变量来写。

在这个管程类中，有两个例程：produce() 和 consume()。生产者线程会反复调用 produce()，把数据放进有界缓冲区；而消费者会反复调用 consume()。这个例子是 Hoare 解法 [H74] 的现代改写版。

你应该注意到，这段代码和前一章基于信号量的方案有一些相似之处。一个主要区别在于，条件变量必须和一个显式的**状态变量**（state variable）配合使用；在这里，整数 `fullEntries` 决定了生产者或消费者是否必须等待，取决于它的状态。相比之下，信号量

```
monitor class BoundedBuffer {
  private:
    int buffer[MAX];
    int fill, use;
    int fullEntries = 0;
    cond_t empty;
    cond_t full;

  public:
    void produce(int element) {
        if (fullEntries == MAX)        // P0 行
            wait(&empty);              // P1 行
        buffer[fill] = element;        // P2 行
        fill = (fill + 1) % MAX;       // P3 行
        fullEntries++;                 // P4 行
        signal(&full);                 // P5 行
    }

    int consume() {
        if (fullEntries == 0)          // C0 行
            wait(&full);               // C1 行
        int tmp = buffer[use];         // C2 行
        use = (use + 1) % MAX;         // C3 行
        fullEntries--;                 // C4 行
        signal(&empty);                // C5 行
        return tmp;                    // C6 行
    }
}
```

图 D.3：**用管程和 Hoare 语义实现的生产者/消费者**

内部有一个数值，起的就是这个作用。因此，条件变量必须和某种外部状态值配对，才能达到同样的目的。

不过，这段代码最重要的地方在于两个条件变量 `empty` 和 `full` 的使用，以及使用它们的 `wait()` 和 `signal()` 调用。这些操作做的正是你所想的事：`wait()` 让调用它的线程在给定的条件上阻塞；`signal()` 唤醒一个正在该条件上等待的线程。

然而，这些调用的工作方式有一些微妙之处；要理解这段代码为什么能工作，理解这些调用的语义至关重要。在操作系统研究者所说的 **Hoare 语义**（Hoare semantics，没错，这个名字取得有点不太走运）下，`signal()` 会立即唤醒一个等待的线程并让它运行；于是，运行中的线程隐式持有的管程锁会立即转交给被唤醒的线程，后者一直运行到阻塞或退出管程为止。注意，等待的线程可能不止一个；`signal()` 只唤醒并运行其中一个，其余的必须等待后续的信号。

举个简单的例子有助于更好地理解这段代码。假设有两个线程，一个是生产者，另一个是消费者。消费者先运行，调用 `consume()`，却发现 `fullEntries = 0`（C0），因为缓冲区里还什么都没有。于是它调用 `wait(&full)`（C1），等待某个缓冲区被填满。接着生

老话：理论与实践
这句老话是：“理论上，理论和实践没有区别；但实践上，有。”当然，只有搞实践的人才会这么告诉你；搞理论的人肯定能证明这句话不成立。

产者运行，发现自己不用等待（P0），把一个元素放进缓冲区（P2），把 fill 索引（P3）和 `fullEntries` 计数（P4）都加一，然后调用 `signal(&full)`（P5）。在 Hoare 语义下，生产者发出信号后不会继续运行；相反，信号会立即把控制权转交给等待的消费者，消费者从 `wait()` 返回（C1），立即消费生产者刚生产的元素（C2），以此类推。只有在消费者返回之后，生产者才能再次运行，从 `produce()` 例程返回。

D.3 理论与实践相遇之处

Tony Hoare 写出了上面的解法，并给出了 `signal()` 和 `wait()` 的确切语义，他是一位理论家。他显然也很聪明；毕竟，快速排序就是他发明的 [H61]。然而，事实证明，这种发信号和等待的语义对于真正的实现来说并不理想。正如那句老话所说：理论上，理论和实践没有区别；但实践上，有。

几年后，施乐帕洛阿尔托研究中心（Xerox PARC）的 Butler Lampson 和 David Redell 在开发一种叫 **Mesa** 的并发语言，并决定用管程作为它的基本并发原语 [LR80]。他们是知名的系统研究者，很快就发现，Hoare 语义虽然更便于证明，却很难在真实系统中实现（原因很多，多到这里恐怕讲不完）。

具体来说，为了做出一个能工作的管程实现，Lampson 和 Redell 决定以一种细微但关键的方式改变 signal() 的含义。signal() 例程现在只被看作一个**提示**（hint）[L83]；它会把一个等待的线程从阻塞状态移到可运行状态，但不会立即运行它。相反，发信号的线程会一直保有控制权，直到它退出管程并被取消调度。

D.4 糟了，有竞态

有了这种新的 **Mesa 语义**（Mesa semantics），我们再来审视一遍上面的代码。再设想一个消费者（消费者 1）进入管程，发现缓冲区是空的，于是等待（C1）。现在生产者来了，填满缓冲区，并发信号表示有一个缓冲区已被填满，于是把等待的消费者从 full 条件变量上的阻塞状态移到就绪状态。生产者继续运行了一会儿，最终让出 CPU。

但是，休斯敦，我们有麻烦了。你看出来了吗？假设另一个消费者（消费者 2）此时调用了 consume() 例程；它会发现缓冲区是满的，于是把它消费掉，然后返回，同时把 fullEntries 设为 0。现在看出问题了吗？问题来了。我们的老朋友消费者 1 终于得以运行，它从 wait() 返回，以为缓冲区是满的（C1……）；不幸的是，情况已经不是这样了，因为消费者 2 抢先溜了进来，在消费者 1 有机会消费之前就把缓冲区消费掉了。因此，这段代码不能正常工作，因为从生产者调用 signal() 到消费者 1 从 wait() 返回的这段时间里，条件发生了变化。下面的时间线说明了这个问题：

生产者	消费者 1	消费者 2
	C0 (fullEnt=0)	
	C1 (Con1：阻塞)	
P0 (fullEnt=0)		
P2		
P3		
P4 (fullEnt=1)		
P5 (Con1：就绪)		
		C0 (fullEnt=1)
		C2
		C3
		C4 (fullEnt=0)
		C5
		C6
	C2 (取用缓冲区，fullEnt=0！)	

图 D.4：**这段代码为什么在 Hoare 语义下不能工作**

幸运的是，从 Hoare 语义换到 Mesa 语义，程序员只需做一点小改动，就能得到能工作的方案。具体来说，线程被唤醒后，应该重新检查它所等待的条件；因为 signal() 只是一个提示，条件有可能已经变了（甚至变了好几次），所以在等待的线程运行时，条件未必处于期望的状态。在我们的例子中，有两行代码必须修改，即 P0 行和 C0 行：

终究不算太难。正因为这样实现起来很容易，如今几乎所有使用带发信号和等待的条件变量的系统，用的都是 Mesa 语义。所以，如果这门课的其他内容你一样都没记住，那就记

```
public:
  void produce(int element) {
    while (fullEntries == MAX) // P0 行（IF 改成了 WHILE）
      wait(&empty);            // P1 行
    buffer[fill] = element;    // P2 行
    fill = (fill + 1) % MAX;   // P3 行
    fullEntries++;             // P4 行
    signal(&full);             // P5 行
  }

  int consume() {
    while (fullEntries == 0)   // C0 行（IF 改成了 WHILE）
      wait(&full);             // C1 行
    int tmp = buffer[use];     // C2 行
    use = (use + 1) % MAX;     // C3 行
    fullEntries--;             // C4 行
    signal(&empty);            // C5 行
    return tmp;                // C6 行
  }
```

图 D.5：**用管程和 Mesa 语义实现的生产者/消费者**

t	Con1	Con2	Prod	Mon	Empty	Full	FE	说明
0	C0						0	
1	C1					Con1	0	Con1 在 full 上等待
2	<上下文切换>					Con1	0	切换：Con1 到 Prod
3			P0			Con1	0	
4			P2			Con1	0	Prod 不用等待（FE=0）
5			P3			Con1	0	
6			P4			Con1	1	Prod 更新 fullEntries
7			P5				1	Prod 发信号：Con1 现在就绪
8	<上下文切换>						1	切换：Prod 到 Con2
9		C0					1	切换到 Con2
10		C2					1	Con2 不用等待（FE=1）
11		C3					1	
12		C4					0	Con2 修改 fullEntries
13		C5					0	Con2 在 empty 上发信号（没有等待者）
14		C6					0	Con2 完成
15	<上下文切换>						0	切换：Con2 到 Con1
16	C0						0	重新检查 fullEntries：0！
17	C1					Con1	0	再次在 full 上等待

图 D.6：**生产者/消费者运行期间各队列的跟踪**

住这一条：**被唤醒后一定要重新检查条件！**说得更简单些，检查条件时要**用 while 循环**，不要用 if 语句。注意，这样做总是正确的，即使你碰巧运行在采用 Hoare 语义的系统上也一样；那种情况下，你只是多此一举地多检查了一次条件而已。

D.5 稍微看看内部实现

为了更好地理解 Mesa 语义为什么更容易实现，我们来多了解一点 Mesa 管程的实现。在他们的工作 [LR80] 中，Lampson 和 Redell 描述了三种不同的队列，一个线程在某一时刻可能处在其中之一：**就绪**（ready）队列、**管程锁**（monitor lock）队列，以及**条件变量**队列。注意，一个程序可能有多个管程类，也可能有多个条件变量实例；每个这样的实例都有一个队列。

对于单个有界缓冲区管程，我们需要考虑四个队列：就绪队列、一个管程队列，以及两个条件变量队列（一个对应 full 条件，一个对应 empty 条件）。为了更好地理解线程库是怎样管理这些队列的，我们来展示在生产者/消费者的例子中，线程是怎样在这些队列之间转移的。

在这个例子中，我们一步步走过这样一种情况：某个消费者被唤醒后，却发现没有东西可消费。来看下面的时间线。左边是两个消费者（Con1 和 Con2）和一个生产者（Prod），以及它们各自正在执行哪一行代码；右边是这个例子中我们跟踪的四个队列的状态：可运行进程的就绪队列、叫作 Monitor 的管程锁队列，以及 empty 和 full 两个条件变量队列。我们还跟踪了时间（t）、正在运行的线程（就绪队列中正在运行的线程两边加方括号），以及 fullEntries 的值（FE）。

从时间线可以看出，消费者 2（Con2）抢先溜了进来，消费了可用的数据（t=9..14），而消费者 1（Con1）从 t=1 起就一直在等待 full 条件被发信号，却没能抢先消费。不过，Con1

 © 2008–23, ARPACI-DUSSEAU

```
monitor class allocator {
    int available; // 还有多少内存可用?
    cond_t c;

    void *allocate(int size) {
        while (size > available)
            wait(&c);
        available -= size;
        // 然后做分配器该做的其他事
        // 并返回一块内存
    }

    void free(void *pointer, int size) {
        // 释放一些内存
        available += size;
        signal(&c);
    }
};
```

图 D.7: **一个简单的内存分配器**

确实被生产者的信号唤醒了（t=7），因此它会再次运行，尽管等它运行时缓冲区已经空了。如果 Con1 不重新检查状态变量 fullEntries（t=16），它就会在没有数据可消费时错误地去消费数据。因此，正是这种自然的实现方式，把我们引向了 Mesa 语义（而不是 Hoare 语义）。

D.6 管程的其他用法

在关于 Mesa 的论文中，Lampson 和 Redell 还指出了几处需要另一种发信号方式的地方。例如，考虑下面这个内存分配器（图 D.7）。

这个例子省略了许多细节，好让我们把注意力集中在唤醒和发信号的条件上。结果发现，上面的发信号/等待代码并不完全正确；你看出原因了吗?

设想有两个线程调用 allocate。第一个调用 allocate(20)，第二个调用 allocate(10)。此时没有可用内存，于是两个线程都调用 wait() 并阻塞。过了一段时间，另一个线程调用 free(p, 15)，释放了 15 字节内存，然后发信号说明它释放了内存。不幸的是，它唤醒的是等待 20 字节的那个线程；那个线程重新检查条件，发现只有 15 字节可用，于是又调用 wait()。而本可以从这 15 字节的释放中受益的线程，也就是调用 allocate(10) 的那个线程，却没有被唤醒。

Lampson 和 Redell 对这个问题提出了一个简单的解决办法。他们不用只唤醒一个等待线程的 signal()，而是改用唤醒所有等待线程的 **broadcast()**（广播）。这样，所有线程都会被唤醒，在上面的例子中，等待 10 字节的线程会发现有 15 字节可用，从而分配成功。

在 Mesa 语义下，使用 broadcast() 总是正确的，因为所有线程被唤醒后本来就应该重新检查自己关心的条件。不过，这样做可能带来性能问题，所以只应在需要时才用。在这个例子中，一次 broadcast() 可能唤醒几百个等待的线程，结果只有一个能成功继续，其余的

```
monitor class Semaphore {
    int s; // 信号量的值

    Semaphore(int value) {
        s = value;
    }

    void wait() {
        while (s <= 0)
            wait();
        s--;
    }

    void post() {
        s++;
        signal();
    }
};
```

图 D.8：**用管程实现信号量**

又立即阻塞；这个问题有时被称为**惊群**（thundering herd），由于额外发生的那些上下文切换，代价很高。

D.7 用管程实现信号量

你大概能看出管程和信号量有很多相似之处。毫不奇怪，可以用其中一个实现另一个。下面我们展示怎样用管程实现一个信号量类（图 D.8）。

可以看到，wait() 只是等待信号量的值大于 0，然后把它减一；而 post() 把值加一，并唤醒一个等待的线程（如果有的话）。就这么简单。

要把这个类当作二值信号量（即锁）来用，只需把信号量初始化为 1，然后在临界区前后放上成对的 wait()/post()。这样，我们就说明了管程可以用来实现信号量。

D.8 现实世界中的管程

前面已经说过，我们用的是“假想的”管程；C++ 并没有这样的概念。下面我们展示怎样写一个类似管程的 C++ 类，以及 Java 怎样用同步方法达到类似的目的。

某种 C++ 管程

下面是用 C++ 和锁、条件变量写成的生产者/消费者代码（图 D.9）。从这个代码示例可以看出，它和前面那个假想的管程代码以及能工作的 C++ 类区别不大。当然，一个明显的区别是显式地使用了一把叫“monitor”的锁。更微妙的是改用了 POSIX 标准的 `pthread_cond_signal()` 和 `pthread_cond_wait()` 调用。尤其要注意，调用 `pthread_cond_wait()` 时，还要传入等待时所持有的那把锁。`pthread_cond_wait()`

```
class BoundedBuffer {
  private:
    int buffer[MAX];
    int fill, use;
    int fullEntries;
    pthread_mutex_t monitor; // 管程锁
    pthread_cond_t empty;
    pthread_cond_t full;

  public:
    BoundedBuffer() {
        use = fill = fullEntries = 0;
    }
    void produce(int element) {
        pthread_mutex_lock(&monitor);
        while (fullEntries == MAX)
            pthread_cond_wait(&empty, &monitor);
        buffer[fill] = element;
        fill = (fill + 1) % MAX;
        fullEntries++;
        pthread_cond_signal(&full);
        pthread_mutex_unlock(&monitor);
    }

    int consume() {
        pthread_mutex_lock(&monitor);
        while (fullEntries == 0)
            pthread_cond_wait(&full, &monitor);
        int tmp = buffer[use];
        use = (use + 1) % MAX;
        fullEntries--;
        pthread_cond_signal(&empty);
        pthread_mutex_unlock(&monitor);
        return tmp;
    }
}
```

图 D.9：**用“管程”实现的 C++ 生产者/消费者**

内部需要这把锁，因为这个线程进入睡眠时必须释放它，返回给调用者之前又必须重新获取它（和管程中的行为一样，只不过这次是用显式的锁）。

Java 管程

有意思的是，Java 的设计者决定采用管程，因为他们认为这是向语言中加入同步原语的一种优雅方式。要使用管程，只需在你想当作管程的方法（或一组方法）上加上关键字 **synchronized**（下面是 Sun 自己文档网站上的一个例子 [S12a, S12b]）：

```
public class SynchronizedCounter {
    private int c = 0;
    public synchronized void increment() {
        c++;
    }
    public synchronized void decrement() {
        c--;
    }
    public synchronized int value() {
        return c;
    }
}
```

图 D.10：**一个带同步方法的简单 Java 类**

这段代码做的正是你所想的事：提供一个线程安全的计数器。由于任一时刻只允许一个线程进入管程，也就只有一个线程能更新“c”的值，从而避免了竞态条件。

Java 与单个条件变量

在最初版本的 Java 中，每个同步类还附带一个条件变量。要使用它，可以调用 **wait()** 或 **notify()**（有时用 notify 这个词代替 signal，但意思是一样的）。奇怪的是，在这个最初的实现中，没有办法拥有两个（或更多）条件变量。你可能已经注意到，在生产者/消费者方案中，我们总是用两个：一个用来通知某个缓冲区已被清空，另一个用来通知某个缓冲区已被填满。

为了理解只提供一个条件变量的局限，我们来设想只用一个条件变量的生产者/消费者方案。假设两个消费者先运行，都卡在等待上。然后，一个生产者运行，填满一个缓冲区，唤醒一个消费者，接着又想再填一个，却发现缓冲区已满（MAX=1）。于是，我们有一个生产者在等待空缓冲区，一个消费者在等待满缓冲区，还有一个之前在等待、因为已被唤醒而即将运行的消费者。

接着，那个消费者运行，消费了缓冲区。可是，它调用 notify() 时，只会唤醒一个在该条件上等待的线程。由于只有一个条件变量，这个消费者可能唤醒的是那个等待的**消费者**，而不是那个等待的生产者。因此，这个方案行不通。

要补救这个问题，同样可以用广播的办法。在 Java 中，可以调用 **notifyAll()** 唤醒所有等待的线程。在这种情况下，消费者会唤醒一个生产者和一个消费者，但那个消费者会发现 fullEntries 等于 0，于是又回去睡眠，而生产者则会继续运行。和往常一样，唤醒所有等待者可能导致惊群问题。

由于这个缺陷，Java 后来加入了一个显式的 Condition 类，从而能更高效地解决这个问题和其他类似的并发问题。

 © 2008–23, ARPACI-DUSSEAU

D.9 小结

我们介绍了管程，这是 Brinch Hansen 以及后来的 Hoare 在 20 世纪 70 年代初提出的一种结构化概念。在管程内部运行时，线程隐式地持有一把管程锁，从而阻止其他线程进入管程，这样就很容易构造出互斥。

我们还介绍了显式的条件变量，它让线程能够 signal() 和 wait()，和我们在前一章信号量中看到的很像。signal() 和 wait() 的语义至关重要；由于所有现代系统都实现的是 **Mesa** 语义，所以为了正确执行，线程必须重新检查它当初因之睡眠的那个条件。因此，signal() 只是一个**提示**，表明有东西发生了变化；确保条件适合继续执行，是被唤醒线程自己的责任。

最后，由于 C++ 不支持管程，我们看到了怎样用显式的 pthread 锁和条件变量来模拟管程。我们还看到了 Java 怎样用它的 synchronized 例程支持管程，以及在这样的环境中只提供单个条件变量的一些局限。

 © 2008–23, ARPACI-DUSSEAU

参考文献

[BH73] “Operating System Principles” Per Brinch Hansen, Prentice-Hall, 1973. Available: http://portal.acm.org/citation.cfm?id=540365 *One of the first books on operating systems; certainly ahead of its time. Introduced monitors as a concurrency primitive.*

[H74] “Monitors: An Operating System Structuring Concept” C.A.R. Hoare. CACM, Volume 17:10, pages 549–557, October 1974. *An early reference to monitors; however, Brinch Hansen probably was the true inventor.*

[H61] “Quicksort: Algorithm 64” C.A.R. Hoare. CACM, Volume 4:7, July 1961. *The famous quicksort algorithm.*

[LR80] “Experience with Processes and Monitors in Mesa” B.W. Lampson and D.R. Redell. CACM, Volume 23:2, pages 105–117, February 1980. *An early and important paper highlighting the differences between theory and practice.*

[L83] “Hints for Computer Systems Design” Butler Lampson. ACM Operating Systems Review, 15:5, October 1983. *Lampson, a famous systems researcher, loved using hints in the design of computer systems. A hint is something that is often correct but can be wrong; in this use, a signal() is telling a waiting thread that it changed the condition that the waiter was waiting on, but not to trust that the condition will be in the desired state when the waiting thread wakes up. In this paper about hints for designing systems, one of Lampson’s general hints is that you should use hints. It is not as confusing as it sounds.*

[S12a] “Synchronized Methods” Sun documentation. http://java.sun.com/docs/books/tutorial/essential/concurrency/syncmeth.html

[S12b] “Condition Interface” Sun documentation. http://java.sun.com/j2se/1.5.0/docs/api/java/util/concurrent/locks/Condition.html

 © 2008–23, ARPACI-DUSSEAU